

CINEVORA — Kế Hoạch Chuyển Đổi Từ CLI Sang Full-Stack Web Application

*Roadmap kỹ thuật: Spring Boot · React · PostgreSQL ·
Docker*

Chuẩn bị bởi Claude (Senior Dev / DevOps / BA) cho Đỗ Thiết Thạch

Tháng 9, 2026

MỤC LỤC

1. Tóm tắt tổng quan (Executive Summary)
2. Đánh giá hiện trạng & Chiến lược tái sử dụng code
 - 2.1. Nguyên tắc cốt lõi
 - 2.2. Bản đồ tái sử dụng (Reuse Mapping)
 - 2.3. Trạng thái đã hoàn thành (tính đến thời điểm lập kế hoạch)
3. Kiến trúc hệ thống đích (Target Architecture)
 - 3.1. Cấu trúc thư mục Monorepo
4. Roadmap chi tiết theo giai đoạn
 - PHASE 0 — Cleanup & Chuẩn bị Repo ✓ (Đã hoàn thành)
 - PHASE 1 — Thiết kế Database (1 tuần)
 - PHASE 2 — Xây dựng Backend (Spring Boot) (2-3 tuần)
 - PHASE 3 — Xây dựng Frontend (React) (2-3 tuần)
 - PHASE 4 — Tích hợp, Migration dữ liệu & Testing (1 tuần)
 - PHASE 5 — DevOps & Deployment (1 tuần)
5. Bảng công nghệ & Markup Language sử dụng trong tài liệu hóa dự án
6. Ước tính Timeline tổng thể
7. Rủi ro & Biện pháp giảm thiểu
8. Bước tiếp theo ngay sau tài liệu này

1. Tóm tắt tổng quan (Executive Summary)

Cinevora hiện là một ứng dụng CLI Java thuần (Pure Java, kiến trúc MVC, lưu trữ File I/O) mô phỏng hệ thống streaming phim kiểu Netflix. Dự án đã hoàn thiện đầy đủ 4 tính chất OOP, thuật toán tự cài đặt (Bubble Sort, Linear Search, Custom Stack cho Undo/Redo), và validate 3 lớp chặt chẽ.

Mục tiêu của kế hoạch này là chuyển đổi Cinevora thành một **Full-Stack Web Application** hoàn chỉnh, có thể public trên Internet, phục vụ cho mục đích **portfolio cá nhân** và luyện tập quy trình phát triển phần mềm chuẩn công nghiệp.

Quyết định công nghệ đã chốt:

Thành phần	Công nghệ	Lý do chọn
Backend	Spring Boot 3.x (Java 21 LTS)	Tận dụng lại toàn bộ kiến thức OOP + code Model/Controller đã viết; hệ sinh thái mạnh nhất cho Java web
Database	PostgreSQL 16	Mã nguồn mở, hỗ trợ ràng buộc dữ liệu mạnh, miễn phí trên hầu hết nền tảng Cloud
ORM	Spring Data JPA + Hibernate	Giảm boilerplate, ánh xạ Entity tự nhiên từ Model cũ
Frontend	React 18 + Vite + TypeScript	Chuẩn công nghiệp phổ biến nhất hiện nay, hệ sinh thái thư viện phong phú
API Style	REST + OpenAPI 3.0 (Swagger)	Chuẩn giao tiếp phổ quát, tự sinh tài liệu API
Auth	Spring Security + JWT	Bảo mật stateless, phù hợp kiến trúc REST
Containerization	Docker + Docker Compose	Đóng gói môi trường nhất quán Dev → Prod
CI/CD	GitHub Actions	Miễn phí, tích hợp sẵn với GitHub đang dùng
Hosting	Render / Railway (BE + DB) và Vercel/Netlify (FE)	Miễn phí ở mức khởi điểm, deploy nhanh, phù hợp portfolio
Quản lý Schema DB	Flyway	Version hóa cấu trúc database, đồng bộ giữa các môi trường
Documentation	Markdown (docs), OpenAPI YAML (đặc tả API), Mermaid (sơ đồ kiến trúc/ERD)	Markup nhẹ, hiển thị tốt trên GitHub, dễ maintain

2. Đánh giá hiện trạng & Chiến lược tái sử dụng code

2.1. Nguyên tắc cốt lõi

Business Logic bất biến theo giao diện. Chỉ tầng Presentation (View) và tầng Persistence (Repository implementation) phụ thuộc vào “hình hài” ứng dụng (CLI vs Web).

Vì Cinevora bản CLI đã được thiết kế đúng theo MVC + Dependency Inversion (Interface Repository) ngay từ đầu, việc “port” sang kiến trúc Web sẽ tái sử dụng được phần lớn công sức đã bỏ ra.

2.2. Bản đồ tái sử dụng (Reuse Mapping)

Tầng cũ (CLI)	Số phần khi lên Web	Đích đến trong kiến trúc mới
<code>model/</code> (User, Admin, Customer, Movie, Category, Statistics...)	✓ Giữ ~95%	<code>backend/entity/</code> — thêm annotation <code>@Entity</code> , <code>@Id</code> , <code>@ManyToOne</code> ...
<code>controller/</code> (MovieController, CategoryController, UserController, CustomerController)	✓ Giữ ~90% logic	<code>backend/service/</code> — đổi tên class thành <code>*Service</code> , đổi cách gọi Repository sang JPA
<code>repository/</code> (Interface: ICategoryRepository...)	✳ Giữ chữ ký, thay implementation	<code>backend/repository/</code> — <code>extends JpaRepository<T, ID></code>
<code>utils/SearchUtils.java</code> , <code>SortUtils.java</code> , <code>FilterUtils.java</code>	✓ Giữ 100% (tùy chọn dùng song song với SQL Query)	<code>backend/util/</code>
<code>utils/CustomStack.java</code>	✓ Giữ 100%, dùng cho Undo/Redo phía Frontend hoặc Redis cache	Tham khảo logic, port sang JS/TS hoặc giữ nguyên Java nếu Undo/Redo xử lý server-side
<code>utils/ValidationException.java</code>	✳ Giữ ý tưởng, mở rộng thành hệ thống Exception Handling	<code>backend/exception/</code> + <code>@ControllerAdvice</code>
<code>utils/ReportExporter.java</code>	✓ Giữ logic CSV RFC 4180	<code>backend/util/</code> — trả file qua <code>ResponseEntity<Resource></code>
<code>utils/InputValidator.java</code> , <code>utils/TextUI.java</code>	✗ Xóa hoàn toàn	Thay bằng Bean Validation (<code>@NotBlank</code> , <code>@Email</code> ...) phía Backend + Form validation phía React
<code>view/</code> (AdminView, CustomerView, LoginView)	✗ Xóa hoàn toàn	Thay bằng REST Controller (JSON) + React Components
<code>Main.java</code>	✗ Xóa, viết lại	<code>@SpringBootApplication</code>
<code>data/*.txt</code>	✳ Dùng làm nguồn seed data	Viết migration script chuyển sang SQL INSERT / Flyway seed

2.3. Trạng thái đã hoàn thành (tính đến thời điểm lập kế hoạch)

- ☒ Tag `v1.0-cli-final` trên bản CLI gốc — mốc nộp bài chính thức được bảo toàn.
 - ☒ Tạo repository mới `cinevora` trên GitHub, giữ nguyên lịch sử commit.
 - ☒ Di dời toàn bộ code CLI vào thư mục `legacy-cli/` để tham khảo trong quá trình port.
 - ☒ Xóa các thành phần CLI-only không có giá trị tái sử dụng: `view/`, `Main.java`, `InputValidator.java`, `TextUI.java`, `run.bat`.
 - ☒ Xác nhận repo local đồng bộ 100% với remote GitHub.
-

3. Kiến trúc hệ thống đích (Target Architecture)



3.1. Cấu trúc thư mục Monorepo

```
cinevora/
├─ legacy-cli/          # Code CLI cũ – giữ tham khảo, KHÔNG deploy
├─ backend/             # Spring Boot project
│   └─ src/main/java/com/cinevora/
│       ├── entity/     # JPA Entity (từ model/ cũ)
│       ├── repository/ # Spring Data JPA Repository
│       ├── service/    # Business Logic (từ controller/ cũ)
│       ├── controller/ # REST Controller (MỚI)
│       ├── dto/        # Request/Response DTO
│       ├── security/   # JWT, SecurityConfig
│       ├── exception/  # GlobalExceptionHandler
│       ├── config/     # CORS, OpenAPI config
│       └─ CinevoraApplication.java
│   └─ src/main/resources/
│       ├── application.yml
│       └─ db/migration/ # Flyway SQL scripts
│   └─ src/test/java/    # Unit + Integration test
│       └─ pom.xml
├─ frontend/            # React project
│   └─ src/
│       ├── pages/
│       ├── components/
│       ├── api/
│       ├── store/      # Zustand/Context
│       ├── routes/
│       └─ App.tsx
│   └─ vite.config.ts
│   └─ package.json
├─ docs/                # Tài liệu dự án (Markdown, ERD, OpenAPI spec)
│   ├── ERD.md (Mermaid diagram)
│   ├── API-SPEC.yaml (OpenAPI 3.0)
│   └─ ARCHITECTURE.md
├─ docker-compose.yml   # Dev environment (backend + frontend + db)
├─ .github/workflows/   # CI/CD pipelines
└─ README.md
```

4. Roadmap chi tiết theo giai đoạn

PHASE 0 — Cleanup & Chuẩn bị Repo ✓ (Đã hoàn thành)

Task	Trạng thái
Tag mốc CLI hoàn chỉnh	✓ Xong
Tạo repo mới, giữ lịch sử Git	✓ Xong
Di dời code cũ vào <code>legacy-cli/</code>	✓ Xong
Xóa thành phần CLI-only	✓ Xong

PHASE 1 — Thiết kế Database (1 tuần)

Mục tiêu: Chuyển đổi mô hình dữ liệu từ File I/O (`.txt`) sang Relational Database chuẩn hóa.

1.1. Thiết kế ERD (Entity Relationship Diagram)

Bảng ánh xạ chi tiết Model cũ → Bảng SQL:

Model cũ	Bảng SQL mới	Thay đổi thiết kế quan trọng
User + Admin / Customer	users	Single Table Inheritance — cột <code>role</code> phân biệt; JPA: <code>@Inheritance(strategy = SINGLE_TABLE)</code>
Category	categories	Thêm <code>created_at</code> , <code>updated_at</code> (audit columns)
Movie	movies	<code>category_id</code> là FK thật (DB tự ràng buộc, không cần validate tay như cũ)
Customer.watchlist (chuỗi M01,M02)	watchlist (bảng N-N)	<code>watchlist(user_id, movie_id, added_at)</code> — chuẩn hóa 3NF
Customer.favouriteList	favourites	<code>favourites(user_id, movie_id, added_at)</code>
Customer.watchHistory	watch_history	<code>watch_history(id, user_id, movie_id, watched_at)</code> — cho phép trùng lặp
WatchProgress	continue_watching	<code>continue_watching(user_id, movie_id, percent, updated_at)</code> — <code>unique(user_id, movie_id)</code>
Statistics	Không cần bảng	Tính bằng SQL aggregate (<code>SUM</code> , <code>AVG</code> , <code>GROUP BY</code>) runtime

1.2. Công cụ quản lý Schema

- Dùng **Flyway** để version hóa migration: `V1__init_schema.sql`, `V2__seed_categories.sql`, `V3__seed_movies.sql` ...
- Viết script Java một lần (tái sử dụng logic parse `|` delimiter từ `MovieRepository.java`, `UserRepository.java` cũ) để đọc `data/*.txt` → sinh ra file `.sql INSERT` cho Flyway seed.

1.3. Checklist Phase 1

- ☐ Vẽ ERD hoàn chỉnh (dùng Mermaid hoặc dbdiagram.io)
 - ☐ Viết `V1_init_schema.sql` (tạo bảng, khóa chính, khóa ngoại, ràng buộc CHECK)
 - ☐ Viết `V2_seed_data.sql` từ dữ liệu mẫu cũ
 - ☐ Test migration chạy được trên PostgreSQL local (Docker container)
-

PHASE 2 — Xây dựng Backend (Spring Boot) (2-3 tuần)

Mục tiêu: Convert toàn bộ business logic CLI thành REST API chuẩn, có bảo mật.

2.1. Khởi tạo dự án

- Dùng `Spring Initializr` với dependencies: `Spring Web`, `Spring Data JPA`, `PostgreSQL Driver`, `Spring Security`, `Validation`, `Flyway`, `Lombok` (tùy chọn), `springdoc-openapi`.
- Java version: 21 (LTS mới nhất, tương thích Spring Boot 3.x).

2.2. Chuyển đổi tầng theo thứ tự (tránh làm rối, làm từ dưới lên)

1. **Entity Layer** — Copy từ `model/`, thêm annotation JPA, giữ nguyên validation trong setter (bổ sung thêm Bean Validation annotation song song để Spring tự validate ở tầng DTO).
2. **Repository Layer** — Tạo interface `extends JpaRepository<Movie, String>`, thêm custom query method (`findByIsActiveTrue()`, `findByCategoryIdAndIsActiveTrue()` ...) thay cho vòng lặp thủ công trong `SearchUtils`.
3. **Service Layer** — Copy nguyên logic từ `controller/` cũ, đổi cách gọi persistence (`movieRepository.save(movie)` thay vì `saveAll(movieList)`).
4. **DTO Layer** — Tạo Request/Response DTO riêng biệt (không trả thẳng Entity ra ngoài — tránh lộ field nhạy cảm như `password`).
5. **Controller Layer (REST)** — Endpoint mới, chỉ gọi Service, trả `ResponseEntity<T>`.
6. **Security Layer** — JWT Authentication Filter + `SecurityConfig` phân quyền theo Role.
7. **Exception Layer** — `@ControllerAdvice` bắt `ValidationException` cũ, convert thành HTTP 400 kèm message.

2.3. Danh sách API Endpoint (rút gọn)

Nhóm	Method	Endpoint	Chức năng
Auth	POST	/api/auth/login	Đăng nhập, trả JWT
Auth	POST	/api/auth/register	Đăng ký Customer
Category	GET/POST/PUT/DELETE	/api/categories	CRUD (Admin only)
Category	PATCH	/api/categories/{id}/restore	Khôi phục soft-delete
Category	GET	/api/categories/trending	Trending Categories
Movie	GET	/api/movies	Danh sách + filter (query params)
Movie	GET	/api/movies/{id}	Chi tiết phim
Movie	POST/PUT/DELETE	/api/movies	CRUD (Admin only)
Movie	GET	/api/movies/ranking	Auto Ranking
Statistics	GET	/api/statistics	Thống kê tổng quan (Admin)
Watchlist	GET/POST/DELETE	/api/users/me/watchlist	Quản lý Watchlist
Favourites	GET/POST/DELETE	/api/users/me/favourites	Quản lý Favourites
History	GET/POST	/api/users/me/history	Lịch sử xem
Continue Watching	GET/PUT/DELETE	/api/users/me/continue-watching	Xem tiếp
Report	GET	/api/users/me/history/export	Xuất CSV

2.4. Bảo mật

- Hash password bằng `BCryptPasswordEncoder` (bắt buộc — dữ liệu mẫu cũ đang lưu plain text, đây là lỗ hổng phải vá trước khi deploy).
- JWT lưu trong `Authorization: Bearer <token>` header.
- `@PreAuthorize("hasRole('ADMIN')")` cho các endpoint quản trị.

2.5. Testing

- Unit test cho Service Layer (JUnit 5 + Mockito) — test lại các rule nghiệp vụ đã có (VD: không cho xóa Category còn Movie active tham chiếu).
- Integration test cho Controller (MockMvc hoặc Testcontainers với PostgreSQL thật).

2.6. Checklist Phase 2

- ☐ Khởi tạo Spring Boot project, kết nối PostgreSQL thành công
 - ☐ Hoàn thành Entity + Repository cho 7 bảng
 - ☐ Hoàn thành Service Layer (port từ Controller cũ)
 - ☐ Hoàn thành REST Controller + DTO cho toàn bộ endpoint
 - ☐ Cấu hình Spring Security + JWT hoạt động (login/register/protected route)
 - ☐ Swagger UI (/swagger-ui.html) hiển thị đầy đủ API
 - ☐ Unit test coverage tối thiểu cho Service Layer
-

PHASE 3 — Xây dựng Frontend (React) (2-3 tuần)

Mục tiêu: Dựng giao diện Web thay thế hoàn toàn CLI Menu, kết nối REST API.

3.1. Khởi tạo dự án

```
npm create vite@latest frontend -- --template react-ts
```

Thư viện chính: - **React Router v6** — điều hướng trang - **TanStack Query (React Query)** — gọi API, cache, loading/error state tự động - **Axios** — HTTP client - **Zustand** — quản lý state nhẹ (Auth, JWT token) - **TailwindCSS** — styling nhanh, responsive - **React Hook Form + Zod** — quản lý form + validate phía client

3.2. Danh sách trang (Page) cần dựng

Trang	Tương ứng chức năng CLI cũ
LoginPage, RegisterPage	LoginView
AdminDashboardPage	AdminView (menu chính)
AdminCategoryPage, AdminMoviePage, AdminUserPage	Sub-menu quản lý
AdminStatisticsPage	Thống kê + Auto Ranking + Trending
BrowseMoviePage	CustomerView.browseMovies()
MovieDetailPage	showMovieDetail()
SearchPage (kèm bộ lọc nâng cao)	searchMovieMenu + advancedFilterMenu
WatchlistPage, FavouritesPage, HistoryPage	Thư viện cá nhân
ContinueWatchingPage	Xem tiếp
RankingPage	Bảng xếp hạng

3.3. Checklist Phase 3

- ☐ Dựng khung layout (Navbar, Sidebar, Protected Route theo Role)
- ☐ Trang Auth (Login/Register) kết nối API thật, lưu JWT
- ☐ Trang Browse/Search/Filter phim hoạt động đầy đủ
- ☐ Trang quản trị Admin (CRUD Category/Movie) hoạt động đầy đủ
- ☐ Trang cá nhân Customer (Watchlist/Favourites/History/Continue Watching)
- ☐ Responsive (Mobile + Desktop)
- ☐ Xử lý lỗi/loading state nhất quán toàn app

PHASE 4 — Tích hợp, Migration dữ liệu & Testing (1 tuần)

Task	Chi tiết
Data Migration	Chạy script chuyển <code>data/*.txt</code> → PostgreSQL qua Flyway seed
Integration Testing	Test toàn bộ luồng: đăng ký → đăng nhập → duyệt phim → thêm watchlist → xem phim → xuất báo cáo
Cross-check nghiệp vụ	Đối chiếu từng rule nghiệp vụ cũ (VD: không xóa Category còn Movie active) đã được giữ đúng khi lên Web
Performance check	Test tốc độ load danh sách phim, tìm kiếm với dataset lớn hơn (nếu có)
Security check	Kiểm tra JWT hết hạn, phân quyền Admin/Customer đúng, chống SQL Injection (JPA đã tự chống phần lớn)

PHASE 5 — DevOps & Deployment (1 tuần)

5.1. Containerization

- Viết `Dockerfile` riêng cho `backend/` (multi-stage build: Maven build → JRE runtime image).
- Viết `Dockerfile` riêng cho `frontend/` (Build → Nginx serve static).
- `docker-compose.yml` cho môi trường Dev: `backend` + `frontend` + `postgres`.

5.2. CI/CD Pipeline (GitHub Actions)

```
.github/workflows/
├── backend-ci.yml    # Build + Test Spring Boot khi push vào backend/
├── frontend-ci.yml   # Build + Lint React khi push vào frontend/
└── deploy.yml        # Deploy tự động khi merge vào main
```

Luồng CI/CD: 1. Push code → GitHub Actions tự chạy `mvn test` (backend) và `npm run build` (frontend). 2. Nếu pass → tự động deploy Backend lên Render/Railway, Frontend lên Vercel. 3. Database migration (Flyway) tự chạy khi Backend khởi động trên môi trường mới.

5.3. Lựa chọn nền tảng Hosting (miễn phí/chi phí thấp, phù hợp portfolio)

Thành phần	Nền tảng đề xuất	Ghi chú
Backend (Spring Boot)	Render hoặc Railway	Có free tier, hỗ trợ Docker trực tiếp
Database (PostgreSQL)	Render Postgres / Supabase / Neon	Free tier đủ dùng cho portfolio
Frontend (React static)	Vercel hoặc Netlify	CDN toàn cầu, deploy tự động từ Git, miễn phí
Domain (tùy chọn)	Namecheap/Cloudflare	Nếu muốn tên miền riêng thay vì subdomain mặc định

5.4. Checklist Phase 5

- ☐ `Dockerfile` build thành công cho cả backend và frontend
- ☐ `docker-compose up` chạy được toàn bộ hệ thống local

- ☐ Pipeline GitHub Actions build + test pass
 - ☐ Deploy thành công Backend + Database lên Render/Railway
 - ☐ Deploy thành công Frontend lên Vercel, trở đúng API Backend
 - ☐ Kiểm tra CORS hoạt động đúng giữa 2 domain khác nhau
 - ☐ Test toàn bộ luồng end-to-end trên bản deploy thật
-

5. Bảng công nghệ & Markup Language sử dụng trong tài liệu hóa dự án

Loại tài liệu	Ngôn ngữ/Định dạng	Lý do
README, tài liệu kỹ thuật nội bộ	Markdown (.md)	Hiển thị đẹp trực tiếp trên GitHub, nhẹ, dễ maintain
Đặc tả API	OpenAPI 3.0 (YAML)	Chuẩn công nghiệp, tự sinh Swagger UI, dễ tích hợp Postman
Sơ đồ kiến trúc, ERD, Flowchart	Mermaid (nhúng trong Markdown)	Render trực tiếp trên GitHub, không cần công cụ vẽ riêng, version control được (là text)
Cấu hình ứng dụng	YAML (application.yml)	Chuẩn Spring Boot, dễ đọc hơn .properties
Giao diện Frontend	JSX/TSX (React)	Kết hợp markup + logic trong cùng component, chuẩn hiện đại
Style	TailwindCSS (utility-class trong HTML/JSX)	Không cần viết file CSS riêng biệt cồng kềnh
Database Schema Migration	SQL (Flyway scripts)	Chuẩn, portable giữa các RDBMS nếu cần đổi sau này

6. Ước tính Timeline tổng thể

Phase	Thời gian ước tính	Ghi chú
Phase 0 — Cleanup	Đã xong	—
Phase 1 — Database Design	~1 tuần	Có thể làm song song với đầu Phase 2
Phase 2 — Backend Development	~2-3 tuần	Phần tốn thời gian nhất, nhưng tái sử dụng nhiều code cũ
Phase 3 — Frontend Development	~2-3 tuần	Có thể bắt đầu sớm với mock data trong lúc chờ Backend hoàn thiện API
Phase 4 — Tích hợp & Testing	~1 tuần	
Phase 5 — DevOps & Deploy	~1 tuần	
Tổng cộng	~7-9 tuần (làm part-time buổi tối/cuối tuần)	Có thể rút ngắn nếu làm Backend và Frontend song song

7. Rủi ro & Biện pháp giảm thiểu

Rủi ro	Mức độ	Biện pháp
Password đang lưu plain text trong dữ liệu mẫu cũ	Cao	Bắt buộc hash bằng BCrypt trước khi migrate vào DB thật, không đưa dữ liệu mẫu gốc lên production
Circular Dependency giữa CategoryController/MovieController (đã có ở bản CLI)	Trung bình	Spring Boot xử lý tốt hơn qua <code>@Lazy</code> hoặc constructor injection kết hợp interface — cần thiết kế lại rõ ràng khi chuyển sang Service Layer
Undo/Redo Watchlist (RAM-based, theo session CLI) khó port nguyên vẹn sang Web (stateless REST)	Trung bình	Cần nhắc: (a) chuyển state Undo/Redo sang lưu ở Frontend (React state, mất khi refresh — chấp nhận được), hoặc (b) dùng Redis nếu muốn bền hơn
Chi phí hosting phát sinh khi vượt free tier	Thấp	Theo dõi usage, free tier Render/Vercel/Supabase đủ cho quy mô portfolio
Thiếu kinh nghiệm Spring Security/JWT lần đầu	Trung bình	Làm theo từng bước nhỏ, test kỹ từng phần (login trước, phân quyền sau), tham khảo tài liệu chính thức Spring

8. Bước tiếp theo ngay sau tài liệu này

1. Thực hiện **Phase 1** — thiết kế ERD chi tiết và viết Flyway migration script đầu tiên (`V1__init_schema.sql`).
 2. Song song, khởi tạo khung `backend/` bằng Spring Initializr với đầy đủ dependencies đã liệt kê ở mục 2.1.
 3. Thiết lập Docker Compose cho môi trường Dev (PostgreSQL container) để bắt đầu code Backend ngay mà không cần cài PostgreSQL trực tiếp lên máy.
-

Tài liệu này là bản kế hoạch sống (living document) — nên cập nhật lại checklist sau mỗi Phase hoàn thành, tương tự cách `CONTEXT.md` đã ghi nhận tiến độ ở bản CLI.